BITS Pilani – Apex Project Report

Project Title: Manufacturing: Autonomous Maintenance Assistant Pipeline

---

Project Metadata

- **Student Name**: Prakhar Tyagi
- **Roll Number**: 2025EB1100149
- **Problem ID**: Manufacturing: Autonomous Maintenance Assistant Pipeline
- **Program**: BITS Pilani BS Program – Apex Project, Trimester 3
- **Submission Date**: July 14, 2026

---

1. Problem Background & Context

Modern manufacturing plants continuously collect large volumes of time-series data from machine sensors, production line status indicators, operator logs, and maintenance histories. This data holds significant potential for proactive maintenance, machine fault prediction, and equipment health monitoring.

However, raw manufacturing datasets frequently contain severe data quality issues that hinder direct analysis or modeling. Typical data flaws include:

- Inconsistent and poorly formatted machine/unit identifiers.
- Missing values and sampling gaps due to transmission dropouts or sensor failures.
- Non-constant sampling frequencies and misalignment between various machines.
- Extreme noise, including impossible physical spikes or sensor malfunctions.
- A lack of explicit remaining useful life (RUL) labels or event flags.

This project implements a structured, robust, and end-to-end data pipeline that addresses these data quality challenges. The pipeline converts noisy, unstructured raw sensor readings into a clean, normalized, and time-aligned feature table suitable for maintenance assistant applications.

---

2. Dataset Description

The pipeline is designed to ingest and clean the **NASA Turbofan Engine Degradation Simulation Dataset (C-MAPSS)**, a standard benchmark dataset in predictive maintenance.

Key Characteristics:

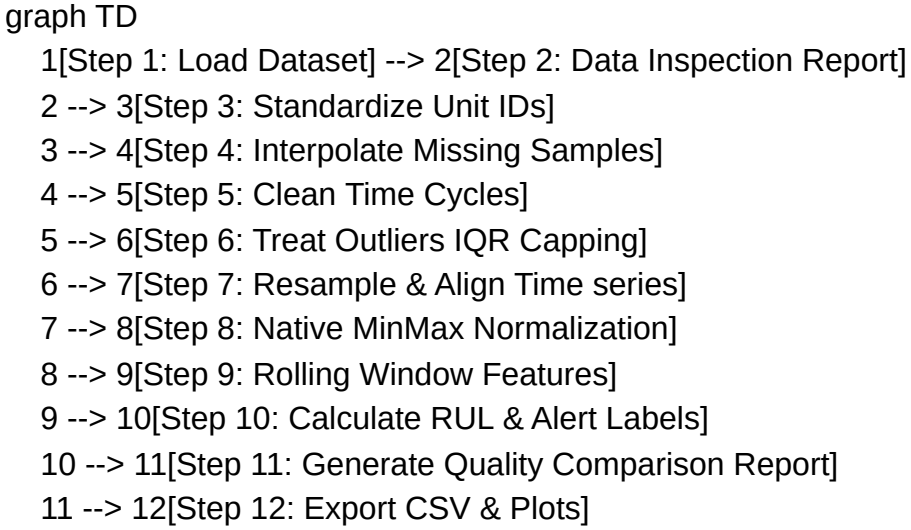
- **Format**: Space-separated text files (.txt).
- **Components**: 15 simulated engine units operating under varying degradation patterns.
- **Key Fields**:
  - **Unit Number**: Identifies the engine instance.

    - **Time Cycle**: Indicates the time step (cycle count).
    - **Operational Settings**: 3 columns representing operational settings.
    - **Sensor Measurements**: 21 columns corresponding to different physical sensor outputs (temperature, vibration, pressure, current, etc.).
- **Data Faults Simulated**: Raw inputs contain random NaNs (1.5% probability), extreme multiplier spikes (0.5% probability), duplicated cycle indices (0.8% probability), inconsistent ID formats (e.g. 'Unit_1', '1.0', '1'), and skipped cycles (gaps).

---

3. The 12-Step Preprocessing Pipeline

The pipeline processes raw records through twelve distinct, chronological stages:



Detailed Breakdown of Steps:

Step	Activity	Technique / Logic
1	**Load Dataset**	Ingests raw files (train_FD001.txt, test_FD001.txt, RUL_FD001.txt). Dynamically downloads them from public sources, with an automatic fallback synthetic data generator if offline.
2	**Data Inspection Report**	Inspects shape, checks missing value counts, variable data types, and min/max cycle ranges across units to diagnose initial flaws.
3	**Standardize Machine Identifiers**	Parses mixed string/float machine identifiers (e.g., 'Unit_1', '1.0') using regular expressions and casts them into standard integers (1).
4	**Handle Missing Values**	Applies linear interpolation on sensor columns grouped by unit_id (maximum gap to interpolate = 5). Leftover edge NaNs are resolved using backward and forward filling.
5	**Clean Cycle Data**	Groups data by unit_id and cycle to find duplicate time cycles. Deduplicates records by taking the mean values and re-sorts data chronologically.
6	**Detect and Treat Outliers**	Implements the Interquartile Range (IQR) method per column per machine. Caps extreme spikes to upper/lower bounds ($Q3 + 3.0 \times IQR$ and $Q1 - 3.0 \times IQR$) to preserve physical readings without discarding records.
7	**Resample and Align Data**	Reindexes the time cycles for each machine from cycle 1 to its maximum recorded cycle. Missing intervals/timestamp gaps are filled and interpolated to align sampling rates.
8	**Normalize Sensor Values**	Standardizes the scales of the 21 sensors to a $[0, 1]$ range using a native MinMax scaling implementation: $X_{norm} = (X - X_{min}) / (X_{max} - X_{min})$. Zero-variance constant sensors are set to 0.0.
9	**Generate Time-Window Features**	Computes rolling window statistics (rolling mean, rolling standard deviation, rolling minimum, rolling maximum) over window sizes of 5 and 10 cycles per machine unit to capture trend histories.
		Calculates Remaining Useful Life (RUL) for each row. Generates a binary warning flag

10	**Integrate Maintenance Indicators**	maintenance_alert (1 if RUL $\le$ 30, 0 otherwise) and a multi-class status level alert_level (2: Critical, 1: Warning, 0: Normal)
11	**Generate Quality Report**	Compares pre-cleaning and post-cleaning statistics (record counts, missing count, outlier count) and saves a comparative Markdown file.
12	**Export Clean Dataset & Analysis**	Writes the final processed datasets to clean CSV files. Generates degradation trend curves and correlation heatmaps as PNG figures.

---

4. Risks & Mitigations

This pipeline mitigates specific time-series data challenges identified in the project outline:

1. **Risk: Missing Sensor Readings (NaNs)**
   Mitigation: Addressed in **Step 4** using localized linear interpolation (gap limit = 5) followed by backward/forward fill to avoid cross-unit leakage.
2. **Risk: Extreme Spikes and Noisy Outliers**
   Mitigation: Addressed in **Step 6** using machine-specific IQR capping. Caps extreme outliers to statistical boundaries ($Q1-3IQR$ / $Q3+3IQR$) without deleting rows or causing time gaps.
3. **Risk: Uneven Sampling Rates & Gaps**
   Mitigation: Addressed in **Step 7** by creating a complete contiguous index range for each machine and linear-interpolating gaps, aligning sampling frequencies across all units.

---

5. Software Tools & Libraries

The system is designed with a lightweight footprint, requiring minimal libraries:

- **Python**: Version 3.10+ (Core language)
- **Pandas**: Version 2.x (Data loading, cleaning, rolling statistics, and reindexing)
- **NumPy**: Version 1.26+ (Array logic and vectorized calculations)
- **Matplotlib**: Version 3.x (Multi-panel sensor trend charts)
- **Seaborn**: Version 0.13+ (Correlation heatmaps)
- **Jupyter Notebook**: Latest version (Interactive step-by-step notebook)

---

6. Execution Instructions

You can run the pipeline automatically or interactively:

Automated Run

Execute the main orchestrator script to run the entire pipeline end-to-end.

Interactive Walkthrough

Open the Jupyter Notebook walkthrough to explore each data processing step interactively and visualize intermediate outputs.